The Wang Method Series

Notebooks #1–#5 (Version 1.0)

Understanding and Solving the 4×4 Rubik's Cube

A Beginner's Guide

Laung-Terng (LT) Wang

August 1, 2026

Acknowledgments

I would like to express my sincere appreciation to the Rubik's Cube community for generously sharing educational videos, tutorials, solving techniques, and ideas through public websites and online communities.

I am especially grateful to the creators of several outstanding educational resources, including Claudia Carlucci, CubeSkills by Feliks Zemdegs, J Perm, EasiestSolve, CubeHead, SpeedyGoneCuber by Raphael "Rafi" Niman, and Noah Richardson. Their high-quality tutorials, clear explanations, and willingness to share their knowledge greatly enriched my understanding of the Rubik's Cube and inspired me to explore simpler, more systematic, and easier-to-understand approaches.

The observations, experiments, classifications, and methods presented in The Wang Method Series were independently developed through my own analysis and engineering perspective. However, they would not have been possible without the open and supportive learning environment created by the worldwide cubing community.

I hope these notebooks will encourage more children to discover that solving the 4×4 Rubik's Cube is not only about learning algorithms, but also about observation, reasoning, experimentation, and the joy of sharing knowledge with others.

Table of Contents

The Wang Method Series

Notebook #1 (Version 1.0)

Understanding the Top-Layer Four-Edge-Pair Patterns

Unusual Top-Layer Four-Edge-Pair Patterns After OLL Parity Correction

Laung-Terng (LT) Wang

August 1, 2026

---

1. Motivation

While solving several 4×4 Rubik's Cubes using the Yau Method followed by reduction, I repeatedly observed unusual top-layer four-edge-pair patterns after correcting the OLL parity and completing the remaining standard 3×3 OLL algorithms.

Instead of encountering only the familiar PLL parity cases, the cube occasionally reached several apparently abnormal four-edge-pair configurations. These patterns did not resemble the standard opposite-edge PLL parity cases commonly encountered during 4×4 cube solving.

These unexpected observations motivated a closer investigation into the relationship between these unusual patterns and the well-known PLL parity.

This notebook records observations made during actual cube solving. The experiments are exploratory and were originally conducted to understand these puzzling patterns. The observations presented here are independent of the particular OLL parity correction algorithm used.

The three experiments presented in this notebook illustrate three representative classes of illegal top-layer four-edge-pair configurations that were later found to account for all **ten illegal states** described in **Notebook #5**.

---

2. Initial Observation

After correcting the OLL parity to restore the yellow cross and subsequently applying standard 3×3 algorithms (such as *Sune*, *A-Perm*, and *Y-Perm*), all corners were solved. However, the four top-layer edge pairs occasionally remained in apparently abnormal configurations.

These configurations appeared unrelated to the familiar opposite-edge PLL parity cases.

This naturally raised the following question:

Are these unusual patterns genuinely new parity cases, or are they simply different visible representations of the same underlying PLL parity?

The following three representative experiments were designed to investigate this question.

---

3. Experiment 1 — A Zero-Aligned Illegal State

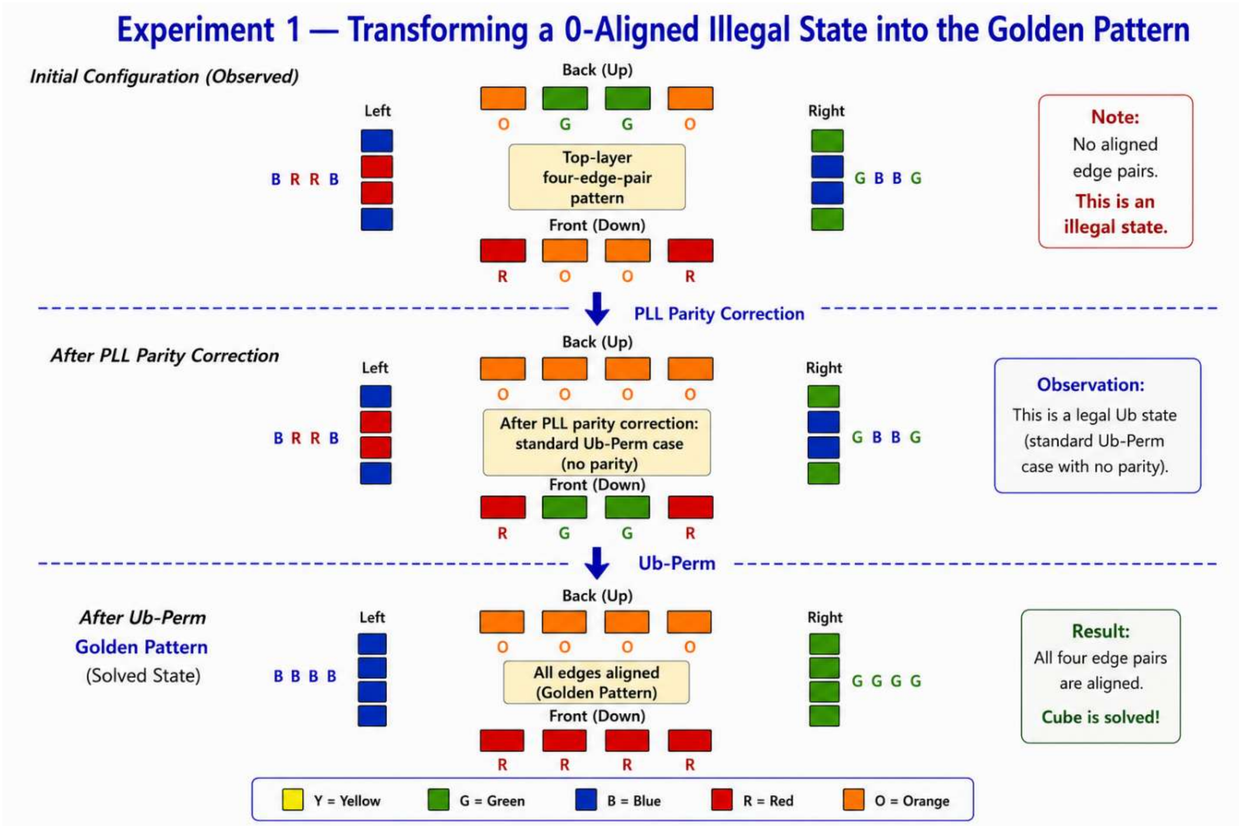

Figure 1.1. A zero-aligned illegal state.

Observed sequence is illustrated in Figure 1.1.

Observation

Applying the standard PLL parity correction transformed the illegal state into an ordinary 3×3 *Ub-Perm* case.

This experiment demonstrates that one class of illegal four-edge-pair configurations can be interpreted as another representation of the underlying PLL parity.

4. Experiment 2 — An Illegal State Transformed by *Z-Perm*

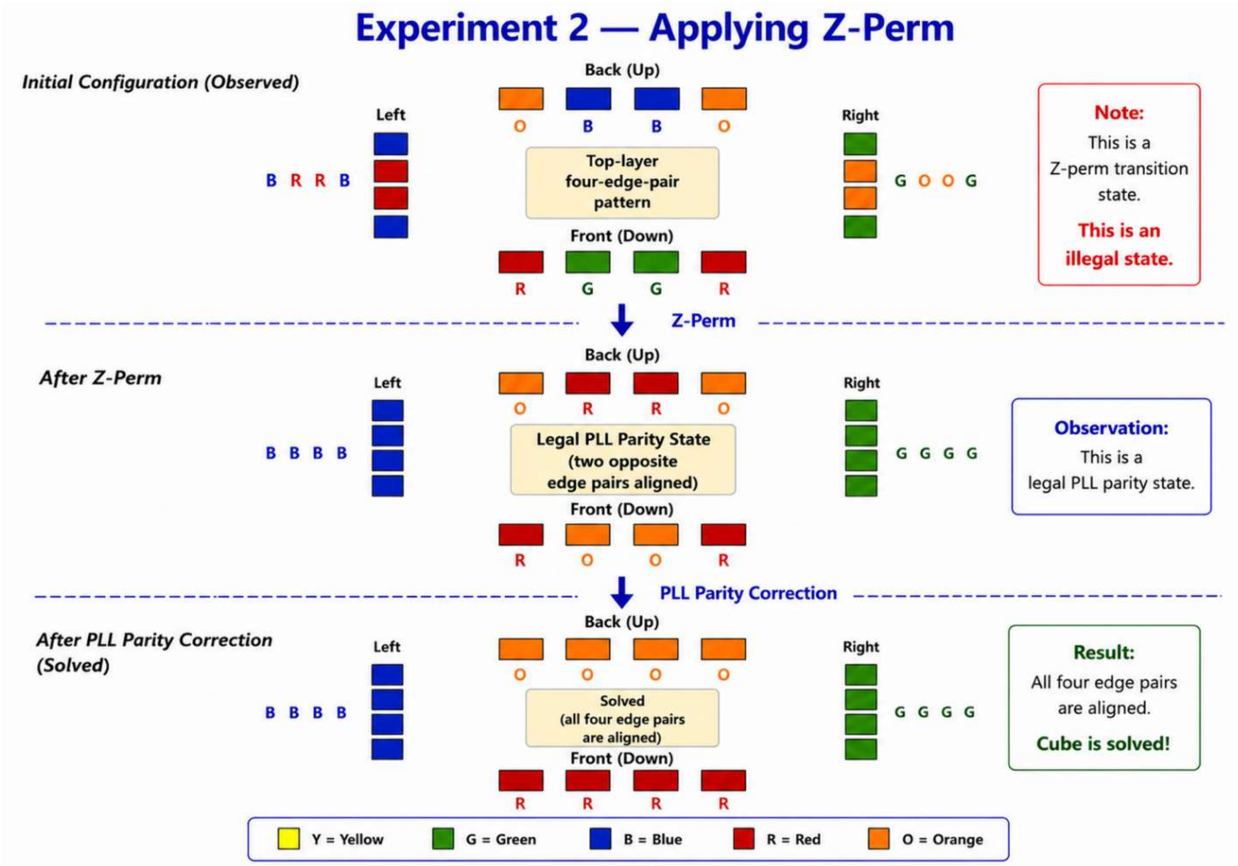

Figure 1.2. An illegal state transformed by *Z-Perm*.

Observed sequence is illustrated in Figure 1.2.

Observation

Applying *Z-Perm* transformed the illegal configuration into a familiar legal opposite-edge PLL parity state.

After the standard PLL parity correction, the cube was solved immediately without requiring any additional PLL algorithm.

5. Experiment 3 — A Two-Adjacently-Aligned Illegal State

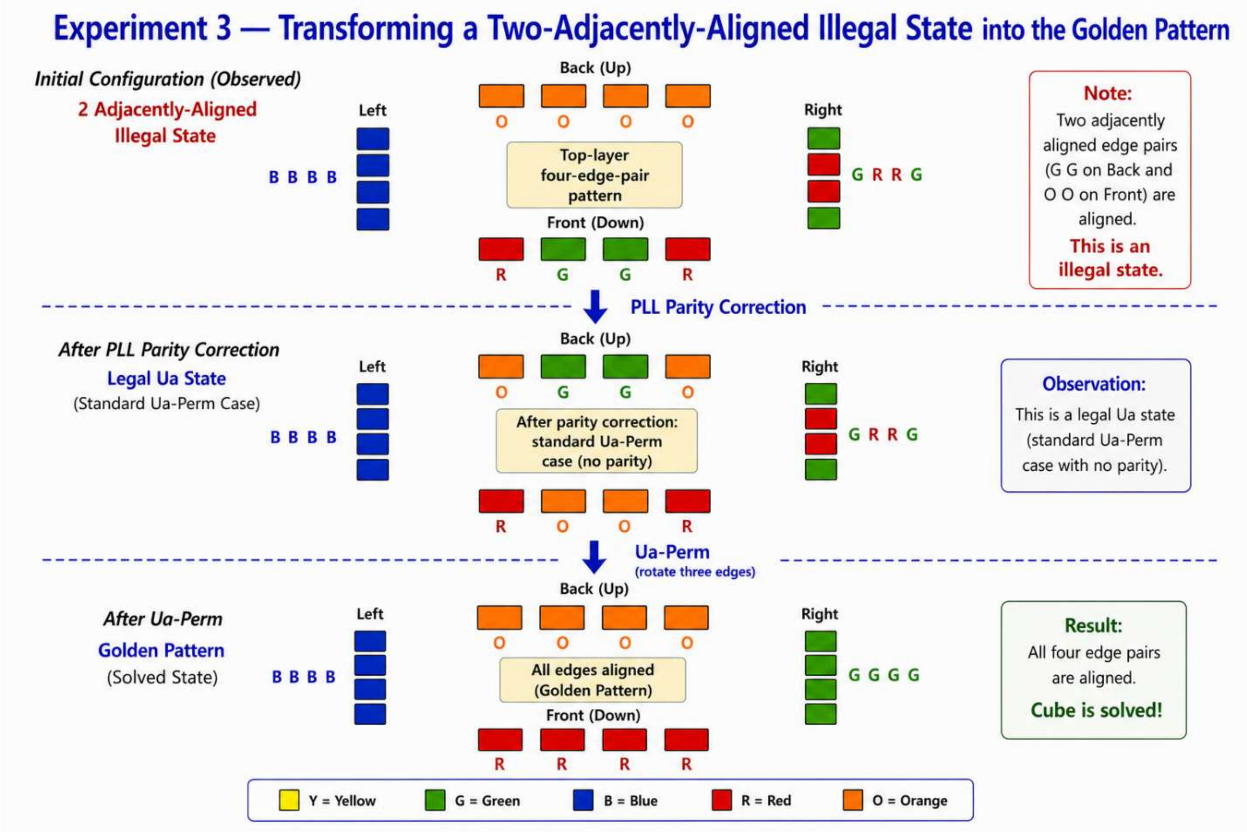

Figure 1.3. A two-adjacently-aligned illegal state.

Observed sequence is illustrated in Figure 1.3.

Observation

Applying the standard PLL parity correction transformed this illegal configuration into an ordinary 3×3 *Ua-Perm* case.

This experiment shows that another seemingly unrelated illegal configuration also corresponds to the same underlying parity phenomenon.

Together, Experiments 1–3 illustrate three representative classes of illegal four-edge-pair configurations.

---

6. Hypothesis

The observed illegal four-edge-pair configurations do not represent new types of parity.

Instead, they appear to be different visible representations of the same underlying PLL parity.

Standard 3×3 PLL algorithms, such as *Z-Perm*, do not eliminate the underlying parity. Instead, they may transform one visible representation of the parity into another.

A PLL parity correction removes the underlying parity and may leave either a legal *Ua-Perm* or a legal *Ub-Perm* case, depending on which opposite edge pair is selected during the correction. The two choices are symmetric.

---

7. Discussion

The three experiments consistently suggest that apparently different illegal four-edge-pair configurations may share the same underlying parity.

During the reduction phase of a 4×4 cube, wide-layer moves allow parity states to be created or eliminated.

Once reduction is complete, however, ordinary 3×3 algorithms appear to preserve the underlying parity while changing only its visible representation.

These observations eventually led to the broader structural classification presented in **Notebook #5**, where all twenty-four top-layer four-edge-pair patterns are classified into six structural groups, and the ten illegal states are organized into two deterministic transition paths.

Although the present notebook is based on experimental observations rather than formal proofs, the results consistently support this interpretation and motivate the structural framework presented in **Notebook #5**.

---

8. Open Questions

Several interesting questions remain for future investigation.

- Can these transformations be explained using permutation-group or graph-theoretic analysis?
- Can illegal configurations be recognized before entering the 3×3 solving phase?

---

9. Lessons Learned

- Not every unusual four-edge-pair pattern represents a new parity.
- Different illegal configurations may represent the same underlying parity.
- PLL parity correction may lead to either *Ua-Perm* or *Ub-Perm*, depending on which opposite edge pair is selected during the correction.
- Ordinary 3×3 algorithms appear to change only the visible representation of the parity.
- These experimental observations motivated the structural framework developed later in **Notebook #5**.

Engineering Perspective

Several seemingly unrelated four-edge-pair configurations were found to share a common underlying parity behavior. These experimental observations eventually led to a structural classification of all twenty-four top-layer four-edge-pair patterns and their deterministic state transitions. Although still exploratory, this engineering perspective provides a logical interpretation of several puzzling parity phenomena encountered during 4×4 cube solving.

The Wang Method Series

Notebook #2 (Version 1.0)

Understanding the Last Three Edge Pairs in the Yau 3-2-3 Pairing Strategy

Revealing the Hidden Four-Wing Cycle

Laung-Terng (LT) Wang

August 1, 2026

---

1. Motivation

The Yau Method is one of the most widely adopted methods for solving the 4×4 Rubik's Cube.

A hallmark of the method is its **3-2-3 edge-pairing strategy**, in which three edge pairs are solved first, followed by two middle edge pairs, leaving only the last three edge pairs to complete.

Although numerous tutorials explain the procedures and algorithms, relatively little attention has been given to **why this strategy works so effectively**.

During a recent study of the last three edge pairs, an unexpected structural pattern emerged.

Rather than appearing random, the remaining four free wings exhibited an underlying cyclic relationship.

This notebook documents that observation.

---

2. Observation

After completing

- the first three edge pairs, and
- the middle two edge pairs,

one aligned edge pair remains intact.

The remaining **four free wings** determine how the last three edge pairs will be completed. Initially these four wings appear to be randomly arranged. However, after a simple slice movement, an unexpected hidden structure becomes visible. This observation motivated the following experiment.

---

3. Experiment 4 — Revealing the Hidden Four-Wing Cycle

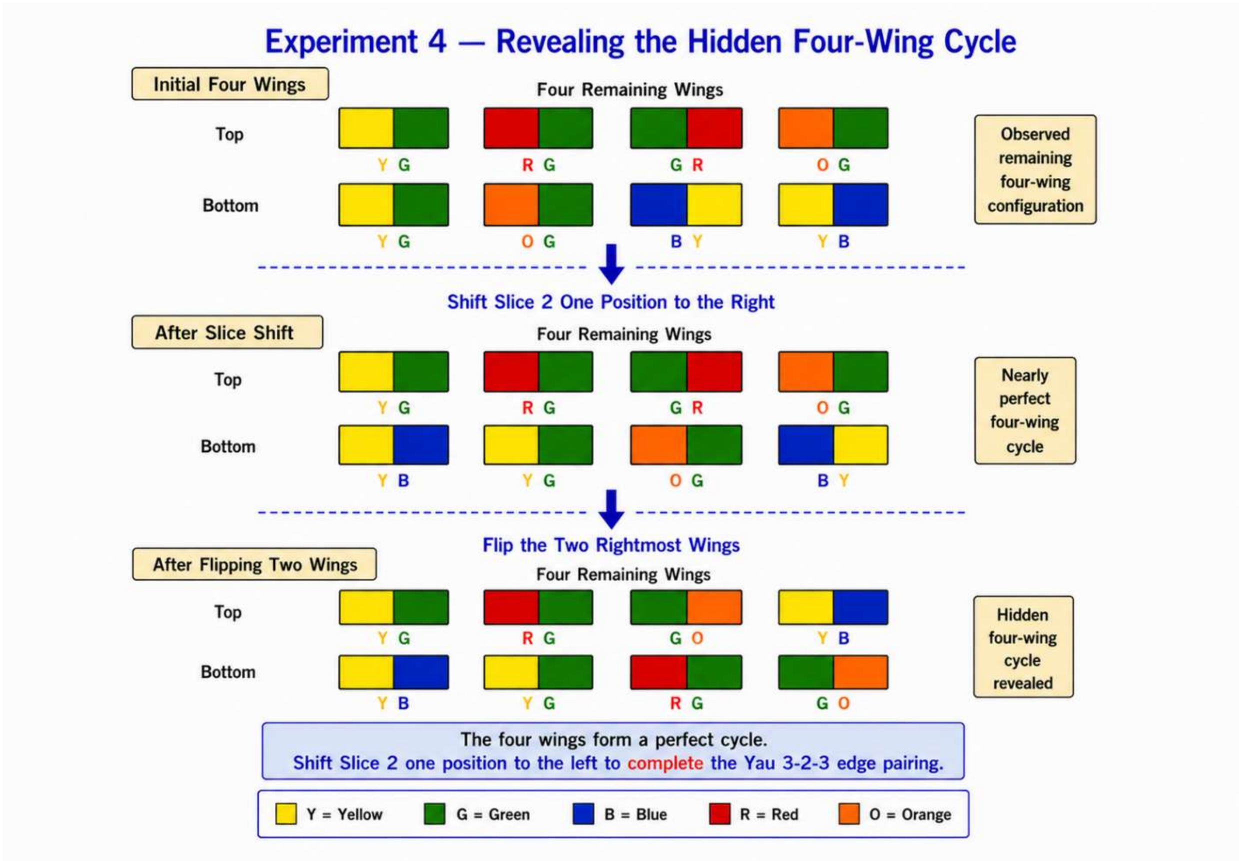

Figure 2.1. Revealing the hidden four-wing cycle.

Observed sequence is illustrated in Figure 2.1.

Initial Four Wings

↓

Shift Slice 2 One Position to the Right

↓

Nearly Perfect Four-Wing Cycle

↓

Flip the Two Rightmost Wings

↓

Perfect Four-Wing Cycle

Observation

Initially, the remaining four free wings appear to have no obvious relationship. This hidden cyclic relationship is not immediately visible in the original configuration.

After shifting Slice 2 by one position, however, they almost form a complete cyclic ordering. Only the orientations of the two rightmost wings remain incorrect.

After flipping these two wings, the four wings become a perfect cycle. The remaining edge pairs can then be completed naturally using the standard Yau procedure. No parity algorithm is involved.

---

4. Hypothesis

Hypothesis

The effectiveness of the Yau 3-2-3 pairing strategy may arise from the fact that, after the first nine edge pairs have been completed, the **remaining four free wings** exhibit a highly constrained cyclic structure.

A simple slice movement appears to reveal this hidden cyclic structure rather than create it.

---

5. Discussion

This observation provides an engineering interpretation of the Yau Method. The slice movement should not be viewed merely as another solving algorithm. Instead, it appears to expose an underlying organization of the remaining four free wings. Once this cyclic relationship becomes visible, only two wing flips are required before the final edge pairs can be completed naturally.

From this viewpoint, the remaining four free wings are not randomly arranged. Instead, they appear to follow a highly constrained cyclic relationship that becomes visible after a simple slice shift.

Although this observation is based on a limited number of experiments, it offers a possible structural explanation for why the final stage of the Yau 3-2-3 pairing strategy is considerably simpler than it first appears.

---

6. Open Questions

Several interesting questions remain for future investigation.

- Does every Yau 3-2-3 pairing case reveal a similar hidden four-wing cycle?
- Can the required slice movement always be predicted?
- How many different four-wing cycle structures exist?
- Can these cycles be characterized mathematically using permutation-group theory?
- Can this structural interpretation lead to simpler or more intuitive solving strategies?

---

7. Lessons Learned

- The last three edge pairs are more structured than they initially appear.
- Slice movements appear to reveal an existing structure rather than create one.
- A hidden four-wing cycle may underlie the final stage of the Yau 3-2-3 pairing strategy.
- Understanding the cube's structure may be more valuable than memorizing additional algorithms.

Engineering Perspective

The effectiveness of the Yau 3-2-3 pairing strategy may arise not only from its algorithms, but also from the hidden structure of the remaining four free wings. Once this cyclic relationship is recognized, the final stage of edge pairing becomes considerably easier to understand.

The Wang Method Series

Notebook #3 (Version 1.0)

Structural Classification of the Last Three Edge Pairs

A Study of Edge Isomorphism and Hierarchical Pattern Recognition

Laung-Terng (LT) Wang

August 1, 2026

---

1. Motivation

When solving the 4×4 Rubik's Cube using the Yau method, the last three edge pairs (**L3E**) often appear in a variety of seemingly unrelated arrangements. At first glance, these configurations appear to require numerous special cases to be memorized.

This notebook shows that the apparent complexity is largely an illusion.

By treating the two edges as **isomorphic**, the remaining three edge pairs exhibit a surprisingly simple structural organization. Rather than viewing each configuration as an independent case, the solver only needs to recognize a small number of structural patterns.

This viewpoint leads naturally to a hierarchical classification of the remaining three edge pairs. More importantly, it provides a unified framework for understanding why different edge-pairing situations often require similar solving strategies, and why certain parity signatures may already become visible during edge pairing.

The purpose of this notebook is not merely to classify the remaining edge-pairing configurations, but to reveal the underlying structural relationships that connect them.

---

2. A Simpler Way to Understand the Last Three Edge Pairs

The last three edge pairs may look confusing at first.

Many cases seem different.

Do we really need to remember every one of them?

No.

Many of these cases differ only in the order of their colors. Once we ignore the color order, many "different" cases become the same.

This notebook will show you how to recognize these hidden patterns.

Let's look at a simple example Figure 3.1 together — Same Edge, Different Looks.

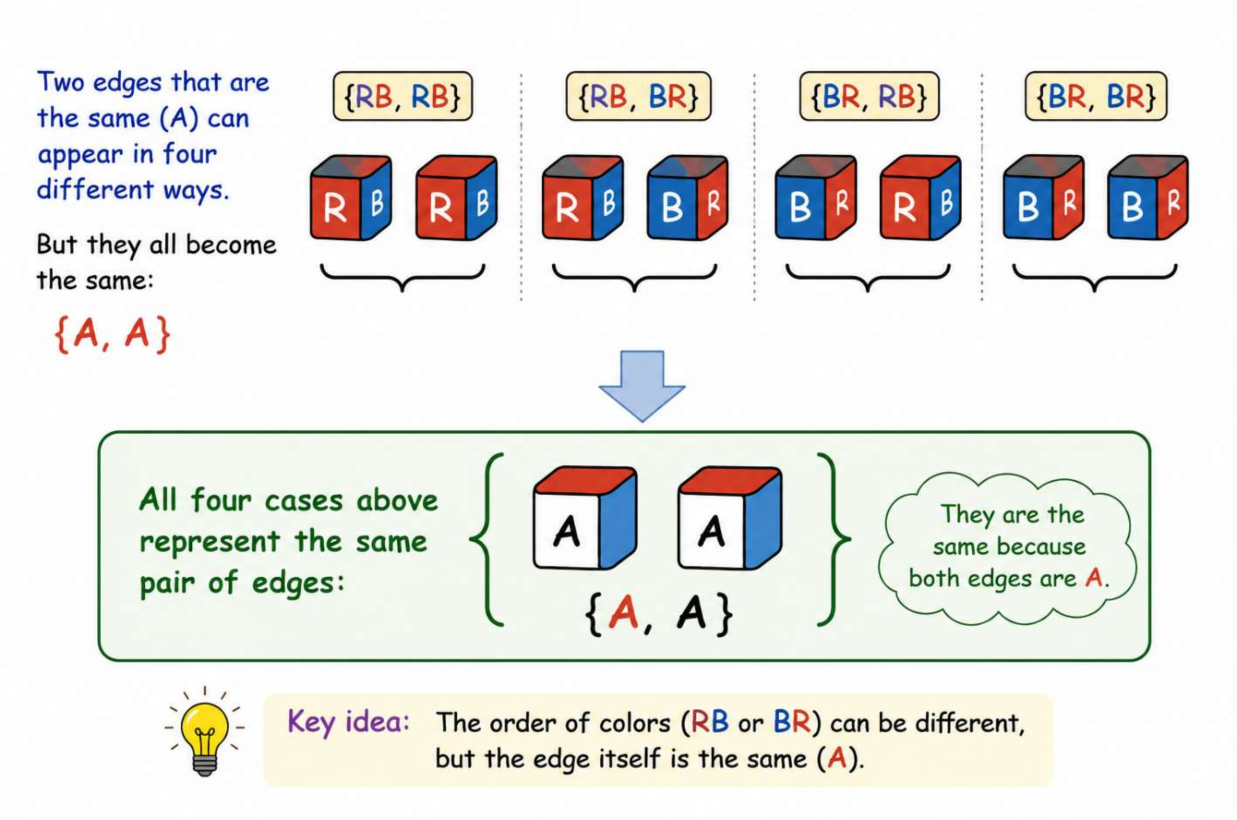

Figure 3.1. Same edges, different looks.

A Red–Blue edge may appear as **RB** or **BR**.

Although the color order is different, it is still the same edge.

So we simply call both appearances **A**.

Now look at two **A** edges.

As shown in Figure 3.1, they can appear in four different ways.

Although these four appearances look different, they all represent the same edge pair:

{A, A}

Instead of remembering every color order, we simply use the letters **A**, **B**, **C**, and **D** to represent the four edge types.

The letter (**A**, **B**, **C**, or **D**) is called the **Edge Identity**.

The color order (such as **RB** or **BR**) is called the **Edge Orientation**.

Although **RB** and **BR** look different, they represent the same edge.

Treating them as the same edge is called **Edge Isomorphism**.

From now on, we will use **Edge Identity** to classify L3E patterns. **Edge Orientation** will be used later when we study parity.

This structural viewpoint also provides the foundation for the parity analysis presented later in this notebook.

3. Hierarchical Structural Recognition

The recognition process begins by inspecting the first two columns. If exactly one pair of identical symbols is present, the configuration immediately belongs to the first cross structural family (Cases 1–4).

If this condition is not satisfied, attention is shifted to the remaining two columns. When an aligned edge pair is present, the orientation of the alignment determines the corresponding structural family. A vertically aligned edge pair belongs to Cases 6–9, whereas a horizontally aligned edge pair belongs to Cases 11–14.

If neither condition is satisfied, the remaining configuration belongs to the diagonal family (Cases 16–19).

Consequently, the sixteen structural cases are recognized through a sequence of simple visual observations rather than by memorizing individual configurations. This hierarchical recognition process greatly reduces the cognitive effort required during edge pairing and makes the structural relationships among different cases much more apparent.

From an educational perspective, this recognition strategy is particularly attractive because beginners can quickly identify the appropriate structural family after only a few seconds of inspection. The emphasis shifts naturally from memorizing numerous cases to recognizing a small number of recurring structural patterns.

---

4. Structural Classification

Based on the concept of edge isomorphism and the hierarchical recognition strategy introduced above, the sixteen structural cases can be organized into four structural families, each representing a distinct structural signature, as illustrated in Figures 3.2 through 3.5.

The following four figures summarize the four structural families together with their corresponding Ideal Cases (Cases 5, 10, 15, and 20). Each figure illustrates the minimum structural and orientation corrections required to transform every structural configuration into its associated Ideal Case

Family I – Cross Structure (Cases 1–4)

The first family is recognized by a unique cross structure. After the aligned edge pair A is shifted to its target position, the other diagonal edge pair B within the first two columns forms a distinctive cross with edge pair A. This cross pattern is unique to Family I.

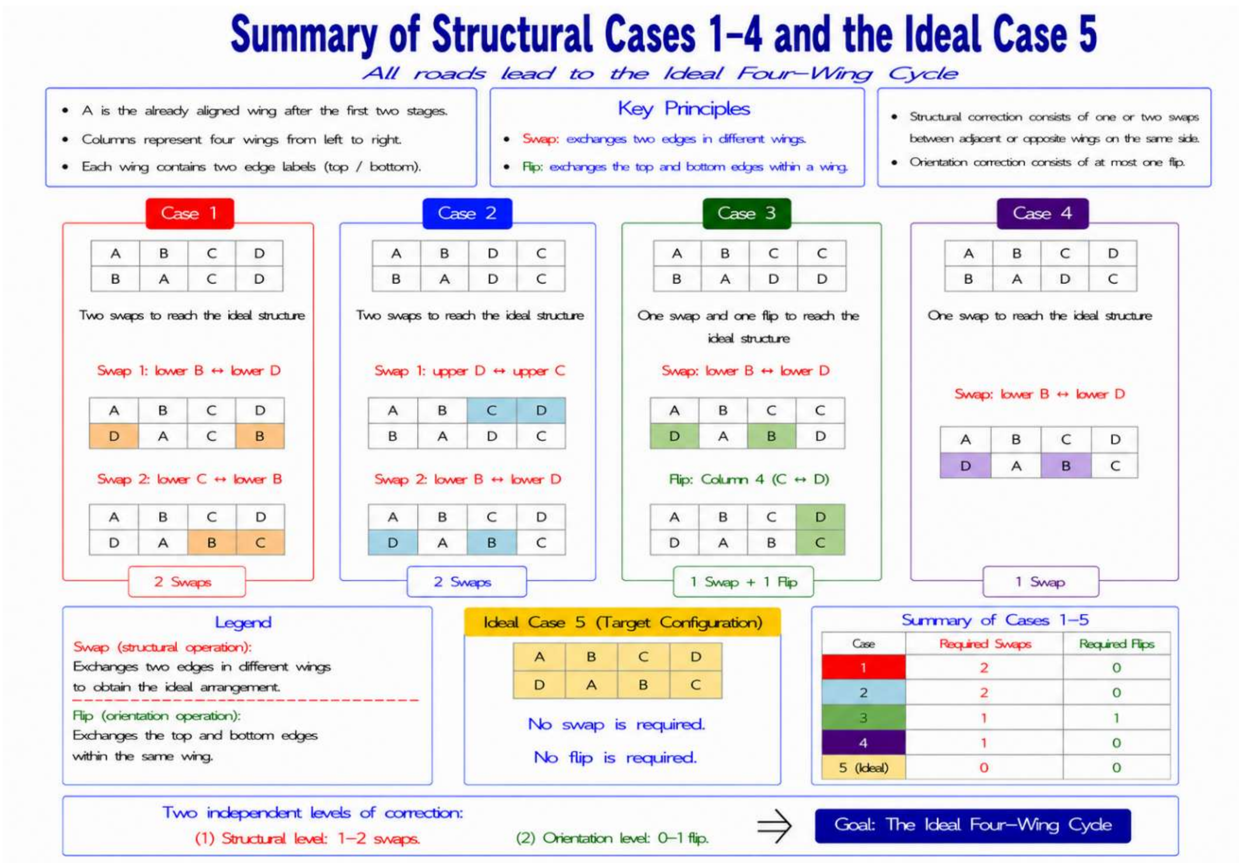

Figure 3.2. Cases 1–4 and the Ideal Case 5.

Family II – Vertical Alignment (Cases 6–9)

The second family is recognized by a vertically aligned identical edge pair within the rightmost two columns. Structural correction requires only one swap, while orientation correction requires only one flip.

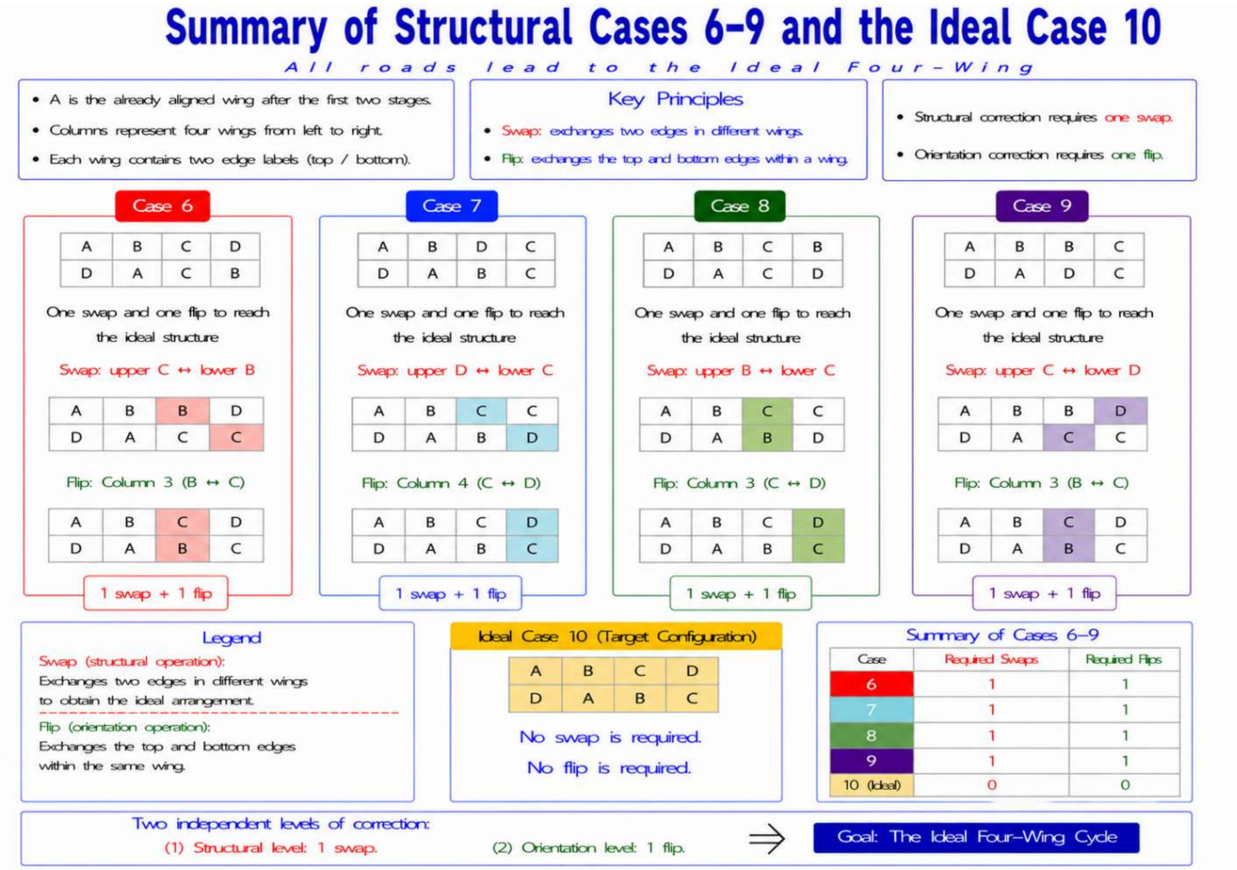

Figure 3.3. Cases 6–9 and the Ideal Case 10.

Family III – Horizontal Alignment (Cases 11–14)

The third family is recognized by a horizontally aligned identical pair within the remaining two rows. Similar to the previous family, structural and orientation corrections remain independent, allowing the required operations to be analyzed separately.

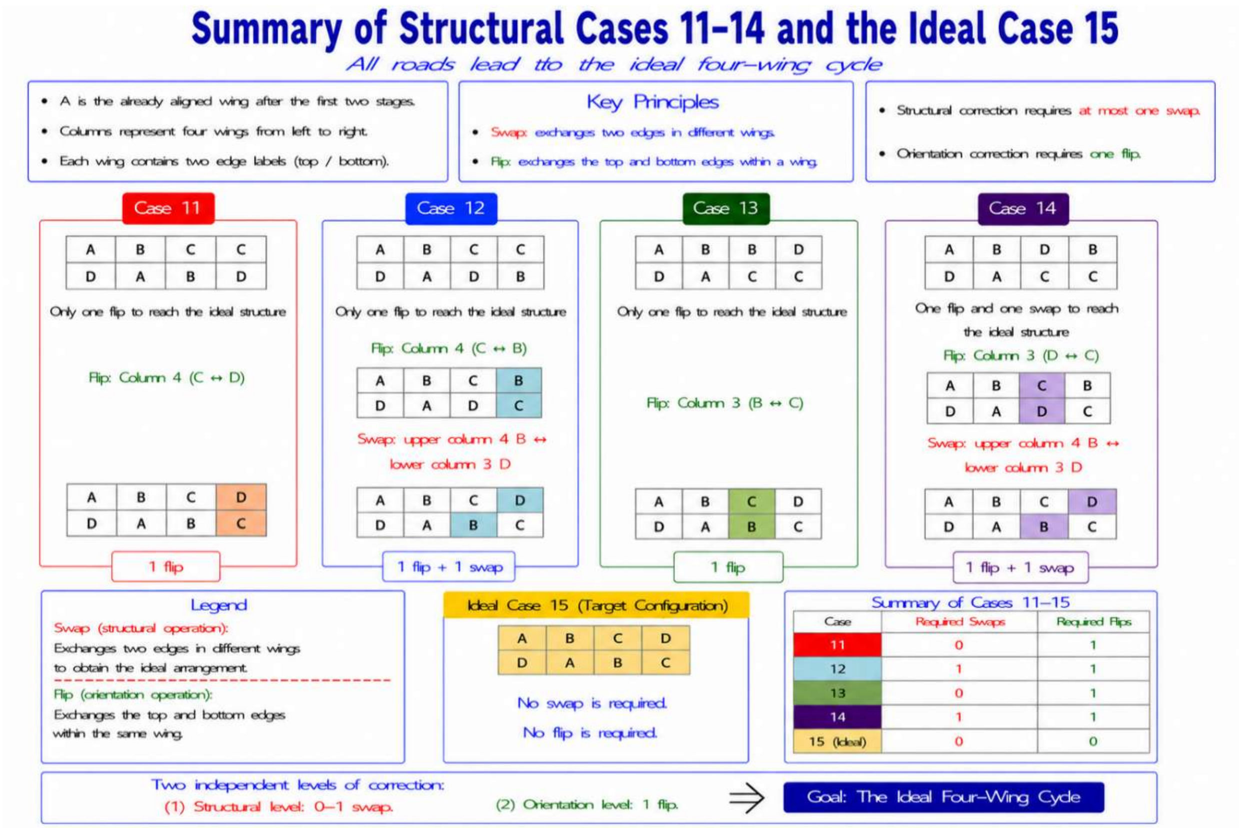

Figure 3.4. Cases 11–14 and the Ideal Case 15.

Family IV – Diagonal Alignment (Cases 16–19)

The fourth family is recognized by diagonally aligned identical symbols. Although this family looks different from the previous ones, the same structural principles continue to apply. Structural correction requires at most one swap, whereas orientation correction requires at most two flips.

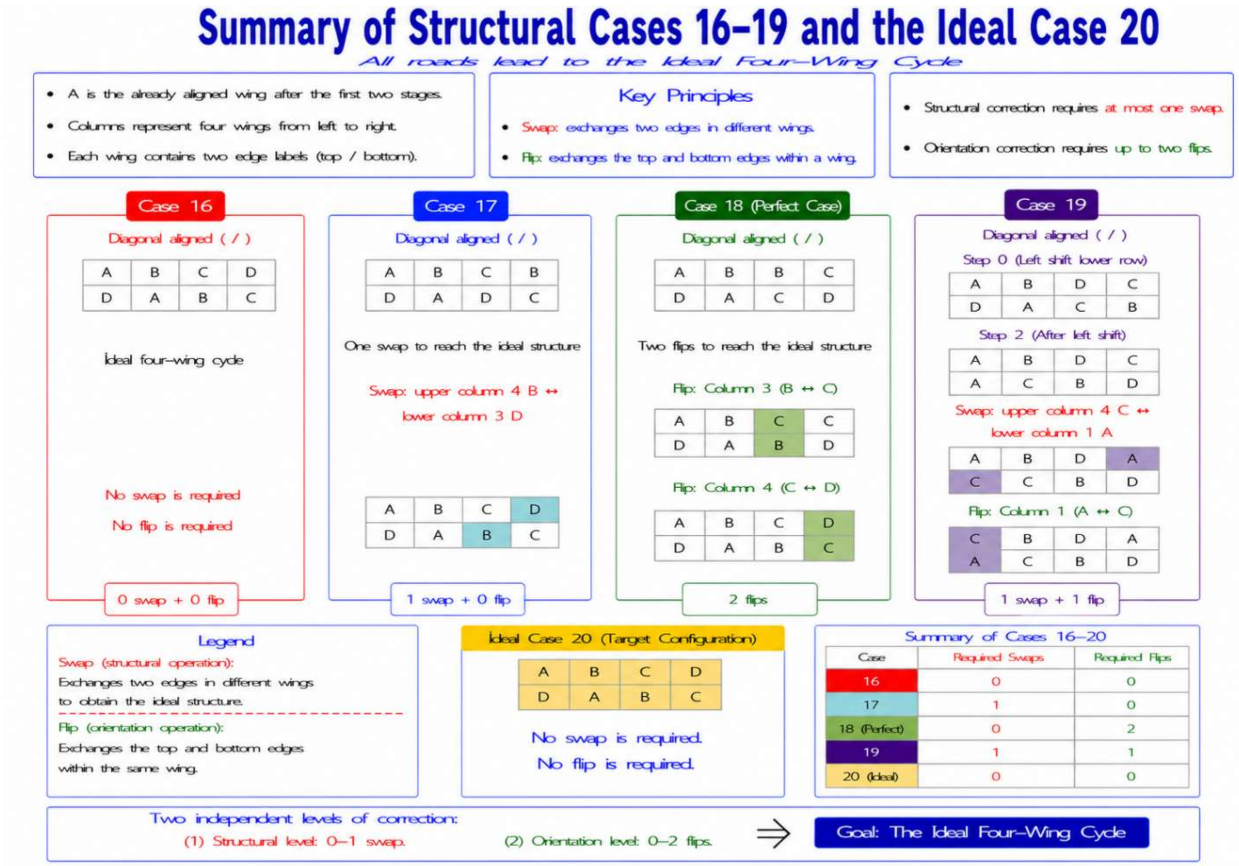

Figure 3.5. Cases 16–19 and the Ideal Case 20.

Although several of these cases in Cases 1–4 initially appear to require one or two structural horizontal swaps, equivalent transformations can often be achieved more efficiently by exploiting diagonal swaps among the four wings. This implementation perspective demonstrates that different structural configurations may share common solving strategies despite their different visual appearances.

Rather than treating the last three edge-pair configurations as unrelated cases, they can be grouped into a small number of structural families based on common structural principles.

Figure 3.6 illustrates that this structural viewpoint not only simplifies classification but also leads naturally to more efficient solution strategies.

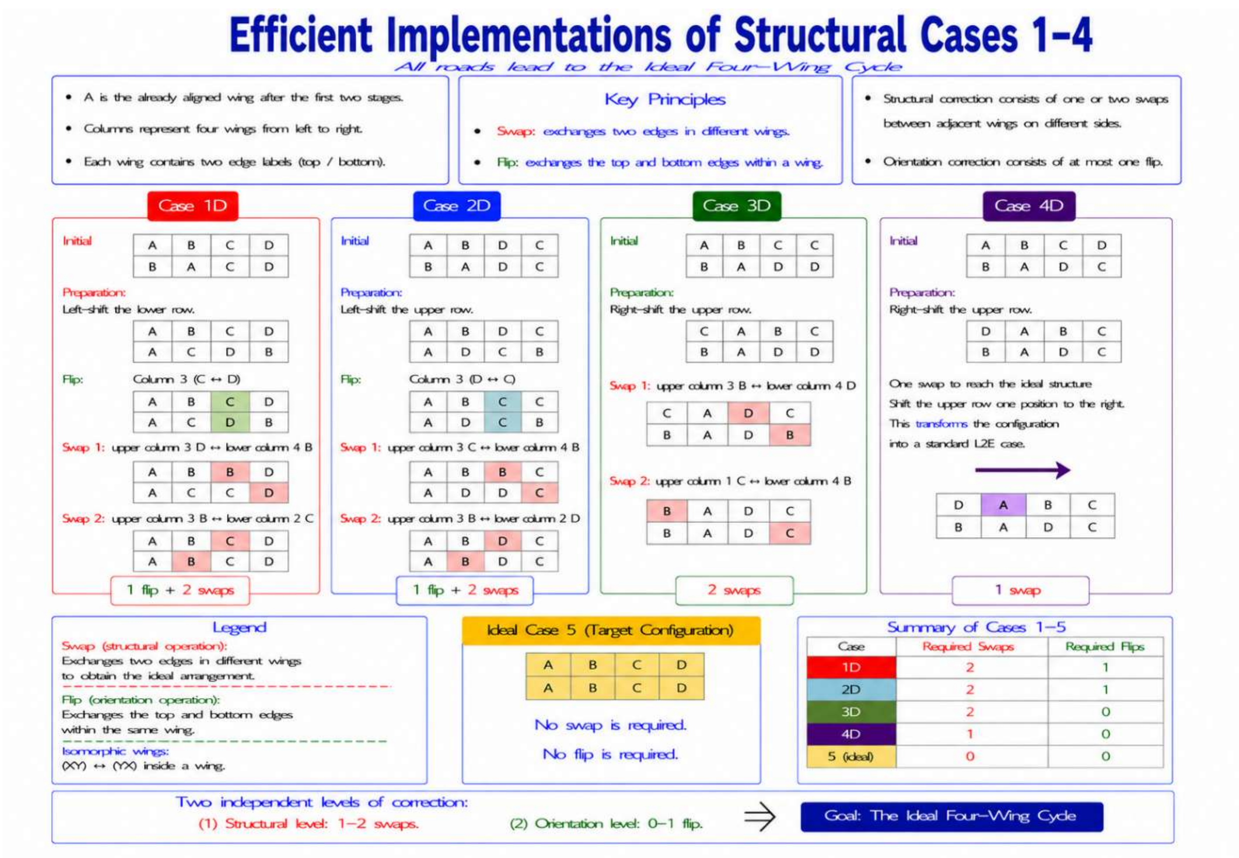

Figure 3.6. Efficient implementations of Cases 1–4.

5. Discussion

The hierarchical structural classification presented in this notebook is a direct consequence of the concept of edge isomorphism. Once the two wings belonging to the same edge are regarded as structurally equivalent, many apparently different edge-pairing configurations become members of the same structural family.

More importantly, the classification emphasizes **recognition rather than memorization**. This significantly reduces the cognitive load on beginners. Instead of remembering numerous independent cases, the solver identifies a small number of structural signatures and then applies the corresponding correction strategy.

The resulting framework also separates structural correction from orientation correction. Structural errors are corrected by swaps, whereas orientation errors are corrected by flips. Because these two types of corrections are largely independent, both the analysis and the implementation become considerably simpler.

From an engineering viewpoint, the classification transforms edge pairing from a collection of isolated cases into a unified structural problem. The emphasis is placed on understanding the underlying relationships among the remaining wings rather than memorizing individual configurations.

---

6. Parity Signatures during Edge Pairing

The concept of edge isomorphism also leads to an interesting observation regarding parity during edge pairing.

During edge reduction, the remaining wing configuration may already exhibit an OLL-parity signature. Because paired wings behave as equivalent representations of reduced edges, one apparent edge pair may remain reversed with respect to the others. This orientation inconsistency corresponds naturally to the well-known OLL parity encountered after edge reduction.

In contrast, PLL parity cannot yet be revealed during edge pairing.

PLL parity concerns the permutation of the completed reduced edges. Before all edge pairs have been completed, the remaining free wings can still be rearranged, allowing incomplete permutation cycles to be completed naturally. Consequently, an apparent exchange between two edge pairs does not yet constitute a genuine PLL-parity state.

This structural interpretation provides a simple explanation for an observation familiar to many experienced solvers: OLL parity may already become visible during edge pairing, whereas PLL parity becomes meaningful only after the reduced-edge permutation has been completely established.

---

7. Preliminary Observations and Open Questions

Based on the experiments completed so far, only one OLL-parity signature has been observed during edge pairing.

No example containing two simultaneous OLL-parity signatures has yet been found. Although this observation suggests that the underlying wing-permutation structure may restrict the cube to at most one OLL-parity signature during edge pairing, no mathematical proof is currently available.

Likewise, no PLL-parity signature has ever been observed before edge reduction has been completed. This observation is consistent with the structural interpretation presented in this notebook, although a rigorous theoretical explanation remains to be established.

Several interesting questions therefore remain open.

- Can the restriction of at most one OLL-parity signature be proved mathematically?
- Can a counterexample containing two simultaneous OLL-parity signatures exist?
- Can the structural classification be derived directly from permutation-group theory?
- Can the concept of edge isomorphism be generalized to larger even-order cubes, such as the 6×6 and 8×8?

These questions suggest several promising directions for future investigation.

---

8. Lessons Learned

Treating two edges as isomorphic provides a remarkably simple framework for understanding the last three edge pairs.

Rather than viewing edge pairing as a large collection of unrelated special cases, the remaining configurations can be recognized hierarchically through a small number of structural signatures. This viewpoint shifts the emphasis naturally from memorization to structural recognition and provides a much simpler way to understand the underlying structural relationships among the remaining edge pairs.

The resulting classification not only organizes the remaining edge-pairing configurations into four structural families but also explains why structurally different cases frequently require similar solving strategies.

The same structural viewpoint also offers a natural interpretation of parity during edge pairing. OLL-parity signatures may already become visible because orientation inconsistencies can already be observed among the partially reduced edges, whereas PLL parity cannot yet emerge because the reduced-edge permutation has not been completely established.

Although many theoretical questions remain open, edge isomorphism provides a simple engineering abstraction that unifies and greatly clarifies the structural behavior of the remaining edge pairs.

Engineering Perspective

Engineering often begins by replacing apparent complexity with an appropriate abstraction.

In this notebook, edge isomorphism serves as that abstraction.

Once the two wings belonging to the same edge are regarded as structurally equivalent, the remaining edge pairs no longer appear as numerous unrelated cases. Instead, they become members of a small number of recognizable structural families governed by common structural principles.

From this viewpoint, edge pairing becomes an exercise in structural recognition rather than case memorization. The same abstraction also explains naturally why OLL-parity signatures may already become visible during edge pairing, whereas PLL parity cannot emerge until the reduced-edge permutation has been completely established.

This notebook therefore represents not only a classification of edge-pairing configurations but also an engineering perspective for understanding their underlying structure.

The Wang Method Series

Notebook #4 (Version 1.0)

Wang's Simple L3E Pairing Method

A Look Ahead Method for Solving the Last Three Edge Pairs of the 4×4 Rubik's Cube

Laung-Terng (LT) Wang

August 1, 2026

---

1. Introduction

After completing the first five edge pairs using the **3-2-3 edge pairing strategy**, the last three edge pairs (**L3E**) remain unsolved. Efficiently solving these last three edge pairs is one of the key ideas of the **Yau Method**.

This notebook presents **Wang's Simple L3E Pairing Method**, a beginner-friendly approach based on **Look Ahead**. Instead of immediately pairing the most obvious edge, which may unintentionally reduce the problem to a last two edge pairs (**L2E**) configuration, the solver first examines the complete L3E configuration and predicts the result of each possible pairing.

Two complementary search paths are used to determine whether an edge can be paired directly or should first be transformed by a **Flip** or **Swap** operation. The selected operation preserves the **L3E** structure so that the solver can continue solving all three remaining edge pairs systematically.

Figures 4.1 and 4.2 summarize the complete Look Ahead decision process.

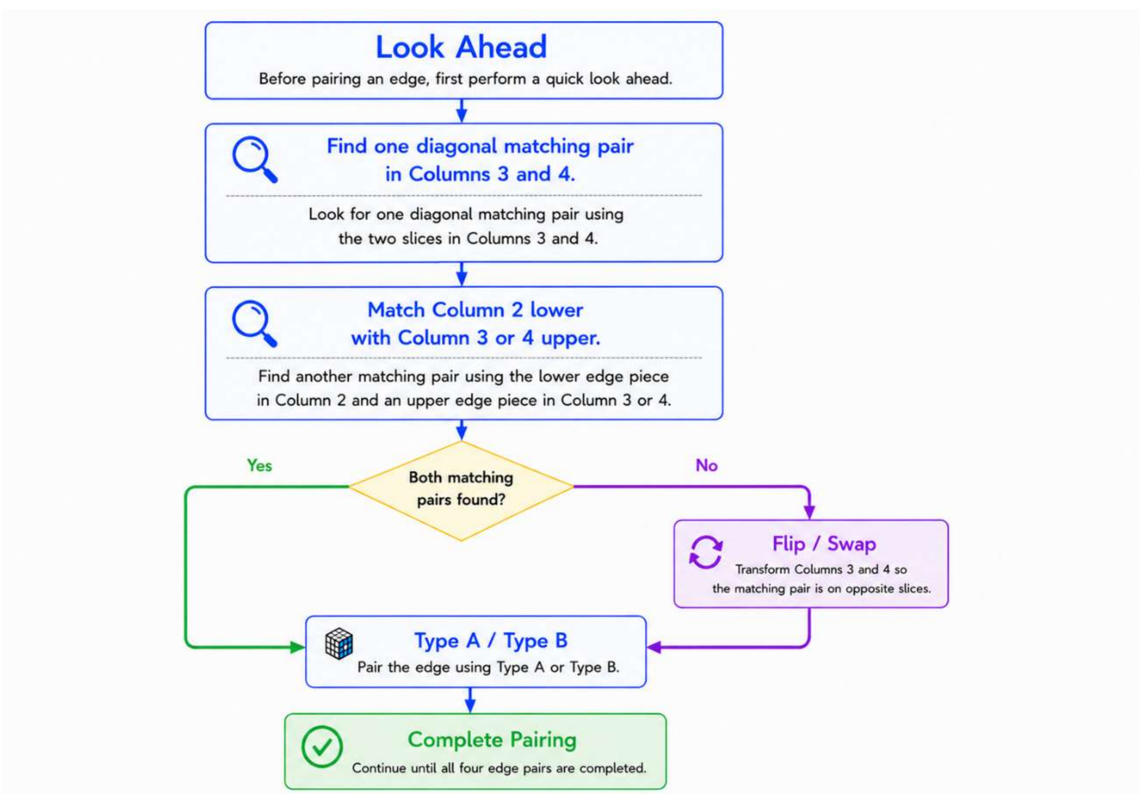

Figure 4.1. Look Ahead flowchart: Search Path 1.

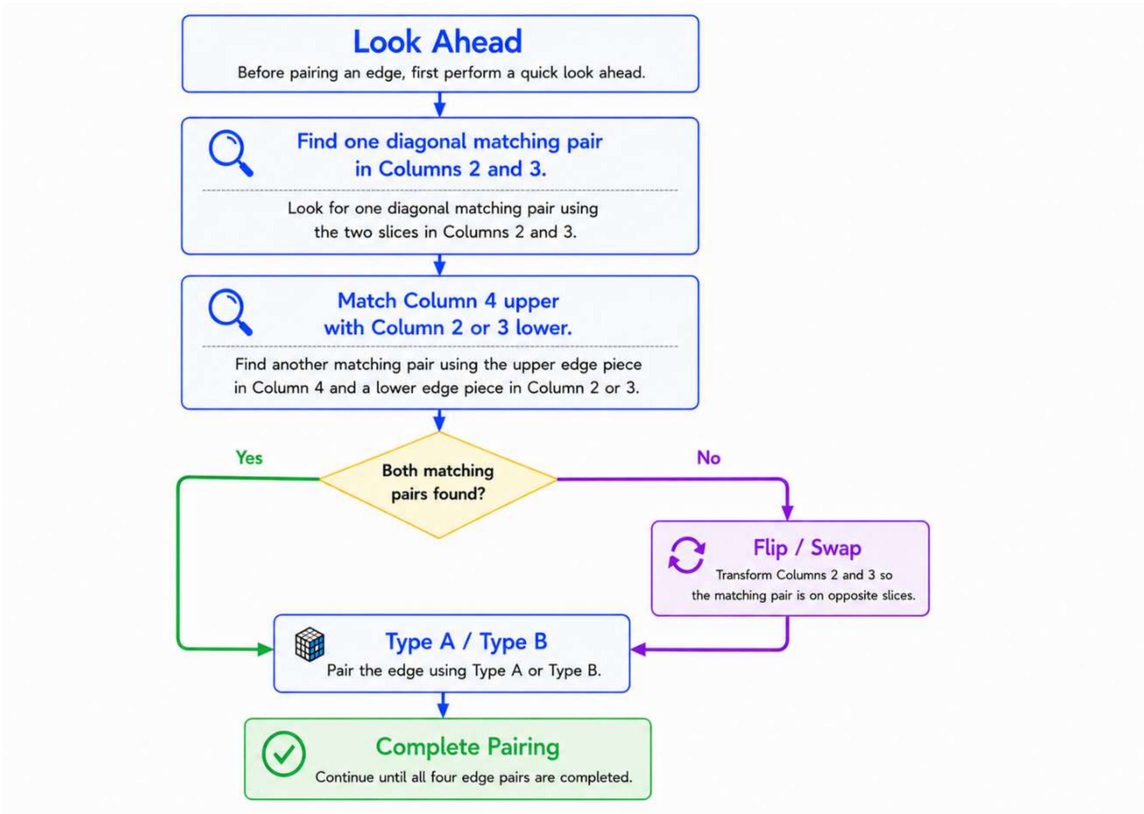

Figure 4.2. Look Ahead flowchart: Search Path 2.

---

2. Look Ahead

Before pairing an edge, first perform a quick **Look Ahead**.

Begin with **Search Path 1** (Figure 4.1).

- Find one diagonal matching pair in **Columns 3 and 4**.
- Then match the lower edge piece in **Column 2** with an upper edge piece in **Column 3 or 4**.

If both matching pairs are found, pair the edge directly using **Type A** or **Type B**.

If Search Path 1 is unsuccessful, try the complementary **Search Path 2** (Figure 4.2).

- Find one diagonal matching pair in **Columns 2 and 3**.
- Then match the upper edge piece in **Column 4** with a lower edge piece in **Column 2 or 3**.

Again, if both matching pairs are found, pair the edge directly using **Type A** or **Type B**.

Otherwise, perform a **Flip** or **Swap**, then complete the pairing using **Type A** or **Type B**.

This Look Ahead strategy considers the complete **L3E** configuration before any pairing operation is performed. Rather than simply solving the immediately visible edge pair, it selects a procedure that preserves the **L3E** structure and avoids unnecessarily inducing an **L2E** configuration.

---

3. Type A Pairing

Type A uses an upper-row shift.

Type A is one of the two fundamental pairing procedures of Wang's Simple L3E Pairing Method. The detailed five-step procedure is illustrated in Figure 4.3. Once Type A is mastered, the remaining configurations can be solved by first transforming them into Type A and then applying this procedure.

The complete Type A pairing procedure requires **five moves**.

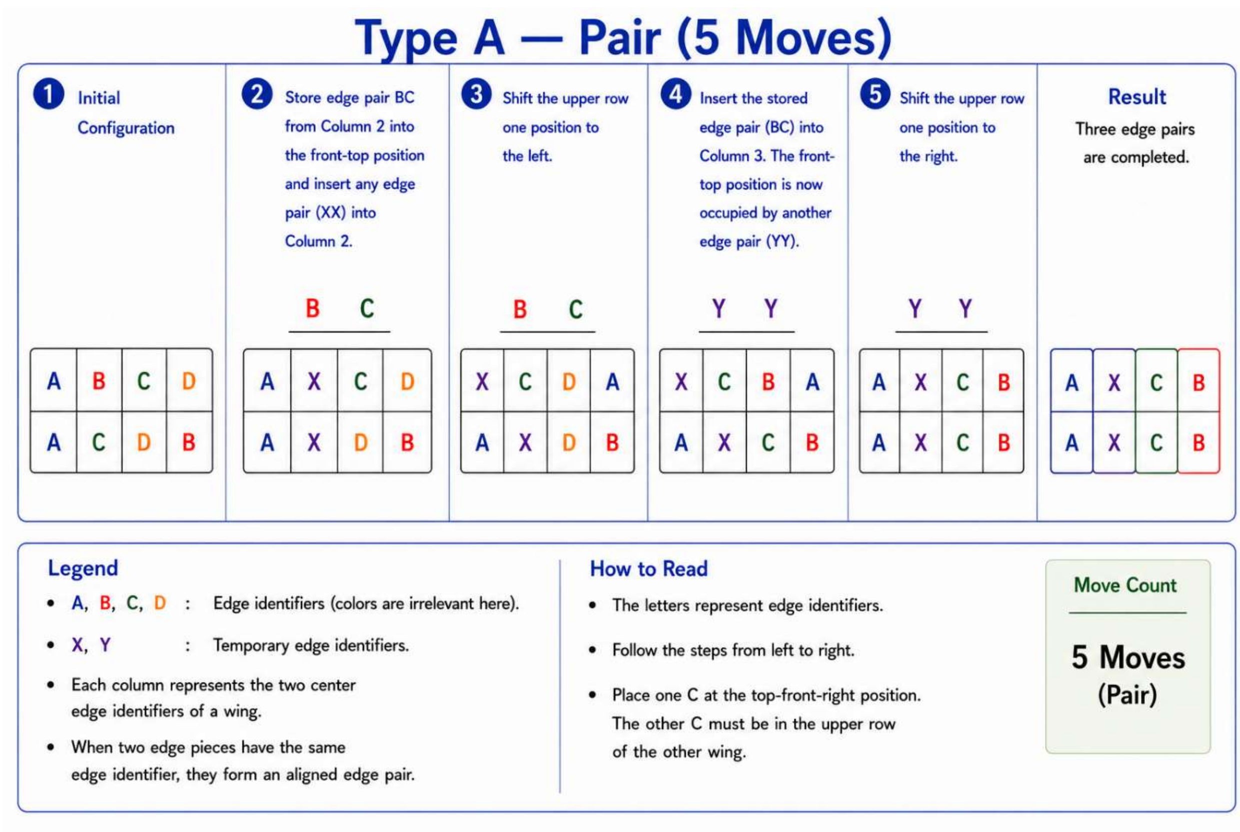

Figure 4.3: Type A — Pair (5 Moves).

4. Type B Pairing

Type B uses the same store-and-insert concept as Type A, but the lower row is shifted instead of the upper row.

Type B is the second fundamental pairing procedure of Wang's Simple L3E Pairing Method. Like Type A, it assumes that the two matching **C edge pieces** have already been positioned on opposite slices.

The complete Type B pairing procedure also requires **five moves**.

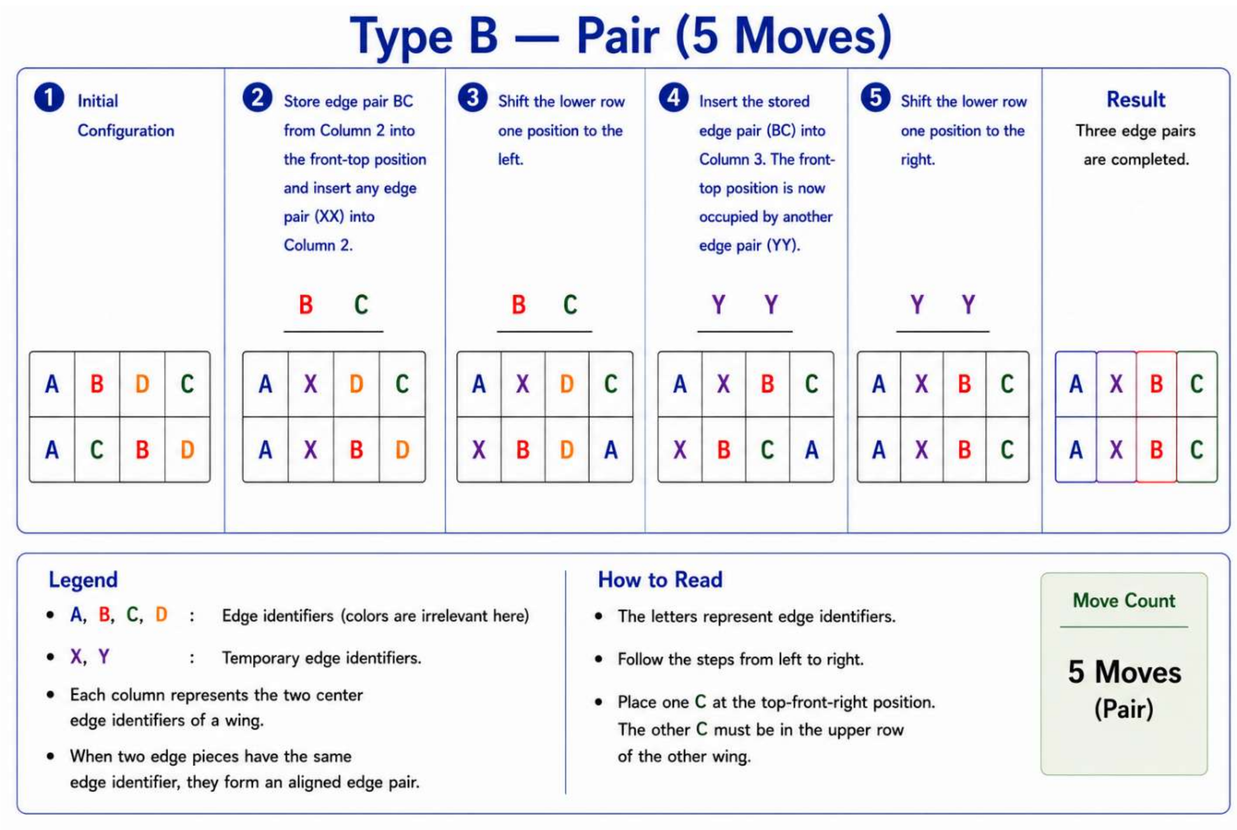

Figure 4.4: Type B — Pair (5 Moves).

---

5. Flip + Pair Transformations

Flip + Pair is the primary transformation used in Wang's Simple L3E Pairing Method.

A flip separates the two matching **C edge pieces** that initially occupy the same slice. The resulting configuration becomes either **Type A** or **Type B**, and can then be solved using the corresponding five-move pairing procedure.

Although a wing can be flipped in fewer moves, Wang's Simple L3E Pairing Method intentionally uses the **seven-move Flip** because it preserves the remaining prepared edge positions. After the last three edge pairs are completed, the solver can proceed directly to the 3×3 stage and begin **CFOP** with the Cross. The shortest move sequence is not always the best solution.

The complete **Flip + Pair** sequence therefore requires **twelve moves**.

5.1 Type A — Flip + Pair

For the Type A transformation, one wing is flipped so that the two matching **C edge pieces** move onto opposite slices. The resulting configuration is identical to the initial configuration of **Type A — Pair**.

The Type A pairing procedure is then applied.

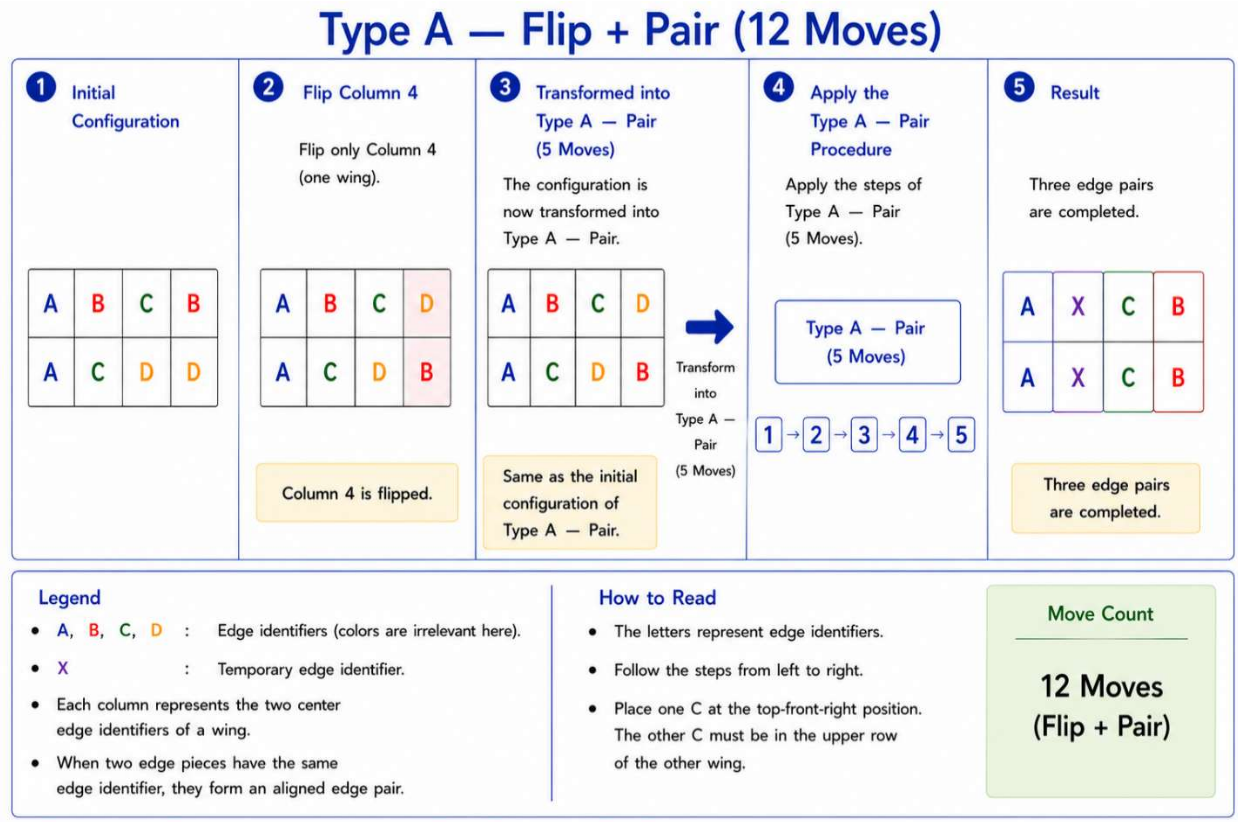

Figure 4.5: Type A — Flip + Pair (12 Moves).

5.2 Type B — Flip + Pair

For the Type B transformation, one wing is flipped to separate the two matching **C edge pieces**. The resulting configuration becomes **Type B — Pair**.

The Type B pairing procedure is then applied.

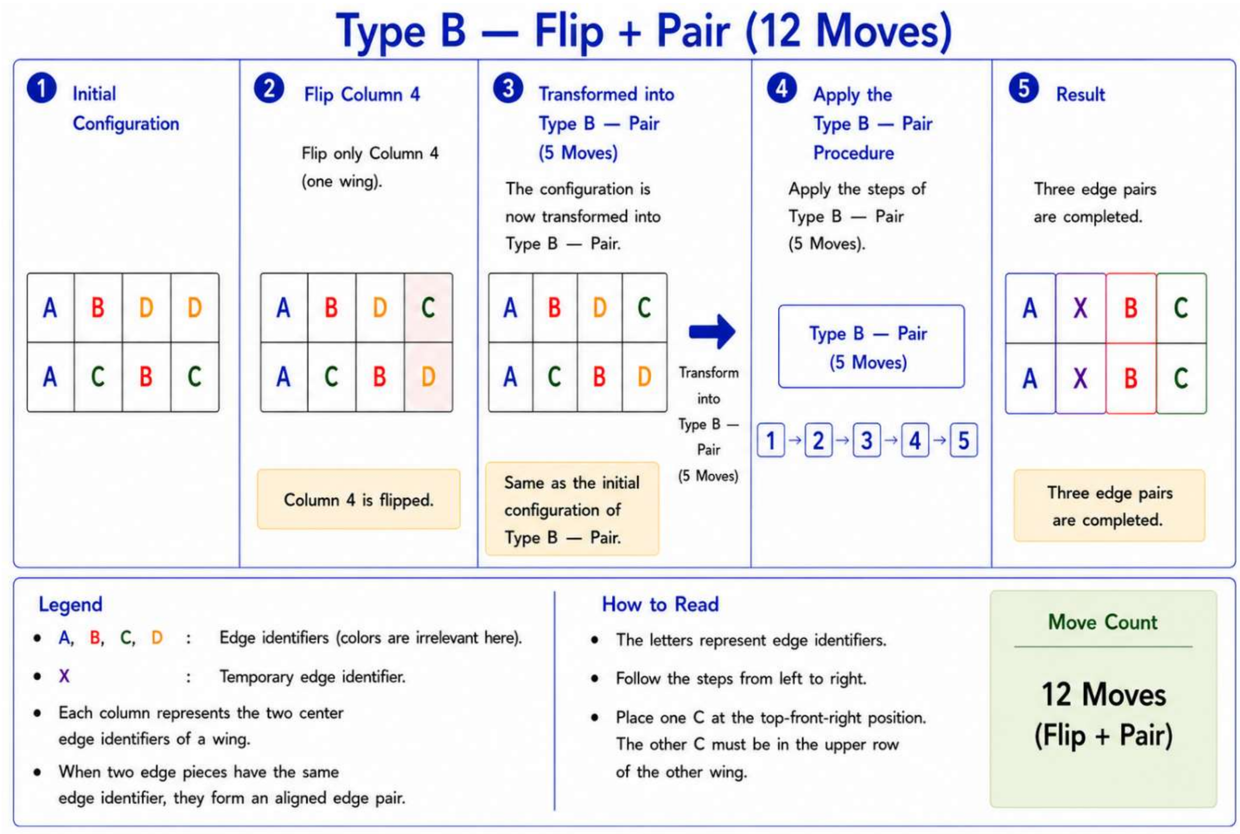

Figure 4.6: Type B — Flip + Pair (12 Moves).

---

6. Type C and Type D Recognition

Type C and Type D represent the two principal same-slice arrangements.

In both types, the two matching **C edge pieces** initially occupy the same slice. Therefore, the solver should not directly complete another edge pair without first transforming the configuration.

6.1 Type C — Flip + Pair

In Type C, flipping the indicated wing transforms the configuration into **Type A — Pair**. The Type A pairing procedure then completes all three edge pairs.

Thus, Type C is solved by:

Flip → Type A — Pair

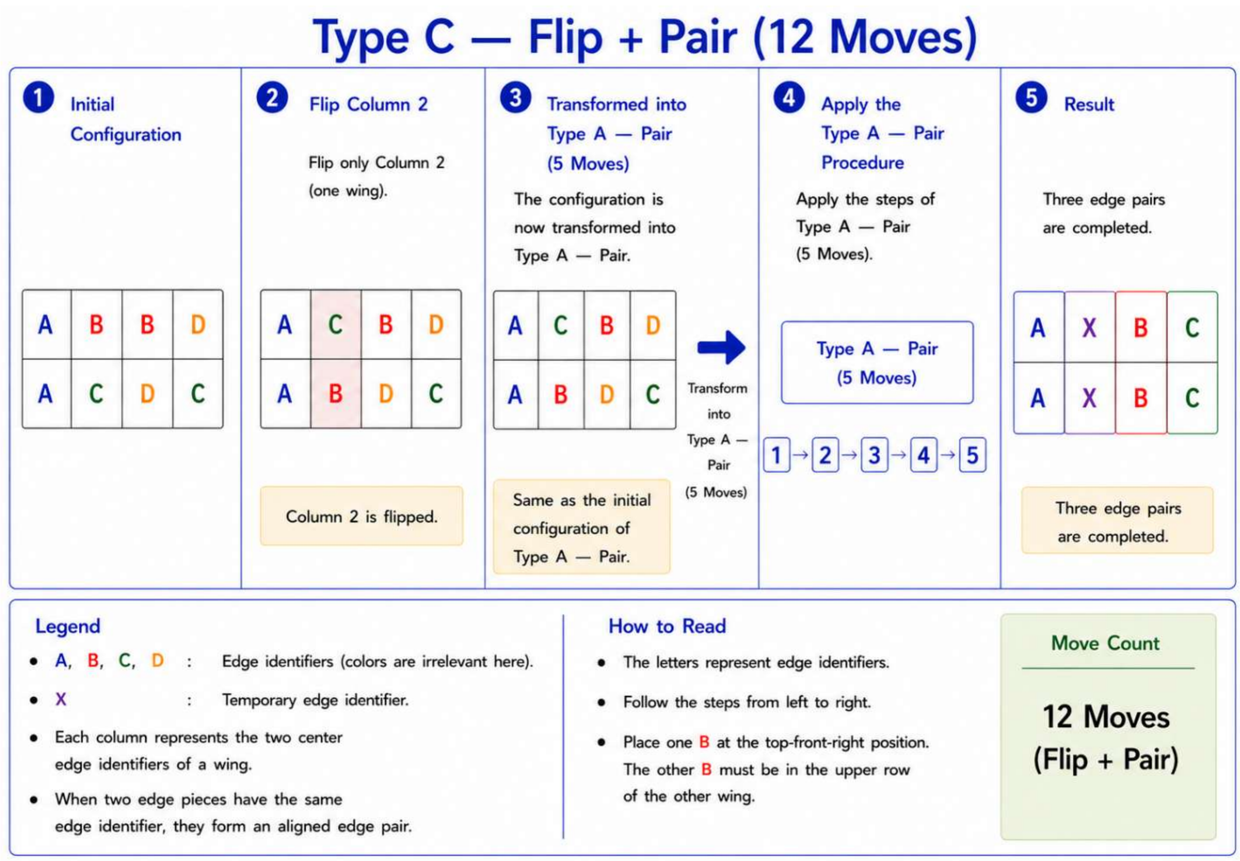

Figure 4.7: Type C — Flip + Pair (12 Moves).

6.2 Type D — Flip + Pair

In Type D, flipping the indicated wing transforms the configuration into **Type B — Pair**. The Type B pairing procedure then completes all three edge pairs.

Thus, Type D is solved by:

Flip → Type B — Pair

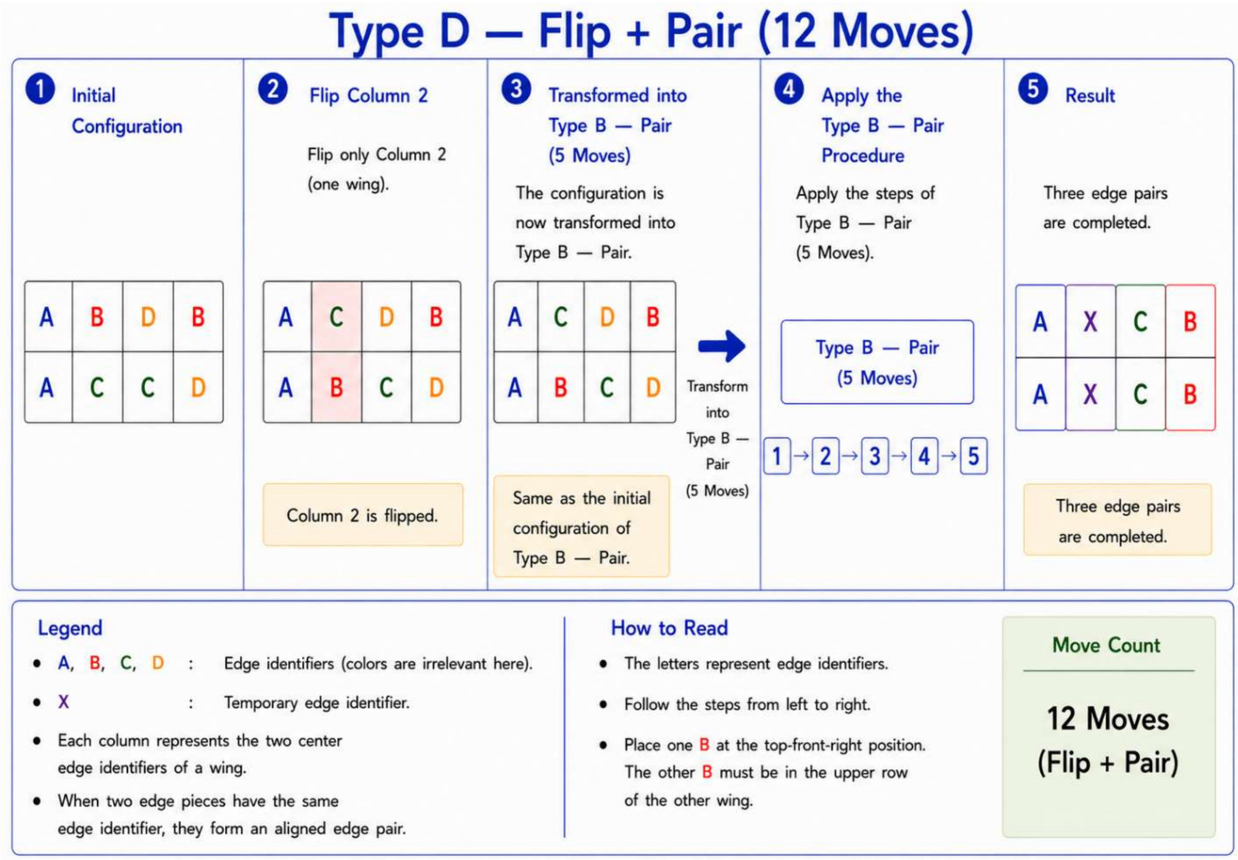

Figure 4.8: Type D — Flip + Pair (12 Moves).

No additional pairing algorithm is required for Type C or Type D.

---

7. Optional Swap + Pair Procedures

Flip + Pair requires only twelve moves and is therefore the preferred procedure for beginners.

However, the shortest move count does not always produce the shortest execution time for an experienced speed cuber.

When the edge pair to be preserved is already conveniently positioned at the front-top, a speed cuber may prefer to perform a Swap instead of a Flip. Although Swap + Pair requires fourteen moves, it may provide a smoother execution with fewer pauses for recognition or regripping.

Like Flip, a Swap can also be performed in fewer moves. However, Wang's Simple L3E Pairing Method intentionally uses the **nine-move Swap** because it preserves the remaining prepared edge positions. After the last three edge pairs are completed, the solver can proceed directly to the 3×3 stage and begin the standard 3×3 **CFOP** method with the Cross.

For this reason, Wang's Simple L3E Pairing Method includes **Swap + Pair** as an optional speed-cubing alternative.

The two approaches serve different purposes:

- **Flip + Pair:** fewer moves and easier for beginners to learn.
- **Swap + Pair:** an optional alternative that may be faster in actual execution when the desired edge pair is already positioned at the front-top.

The choice should therefore be based not only on move count, but also on the current cube position and the solver's execution preference.

7.1 Type A — Swap + Pair

In Type A, swapping the indicated edge pieces transforms the configuration into **Type A — Pair**. The corresponding five-move pairing procedure then completes all three edge pairs.

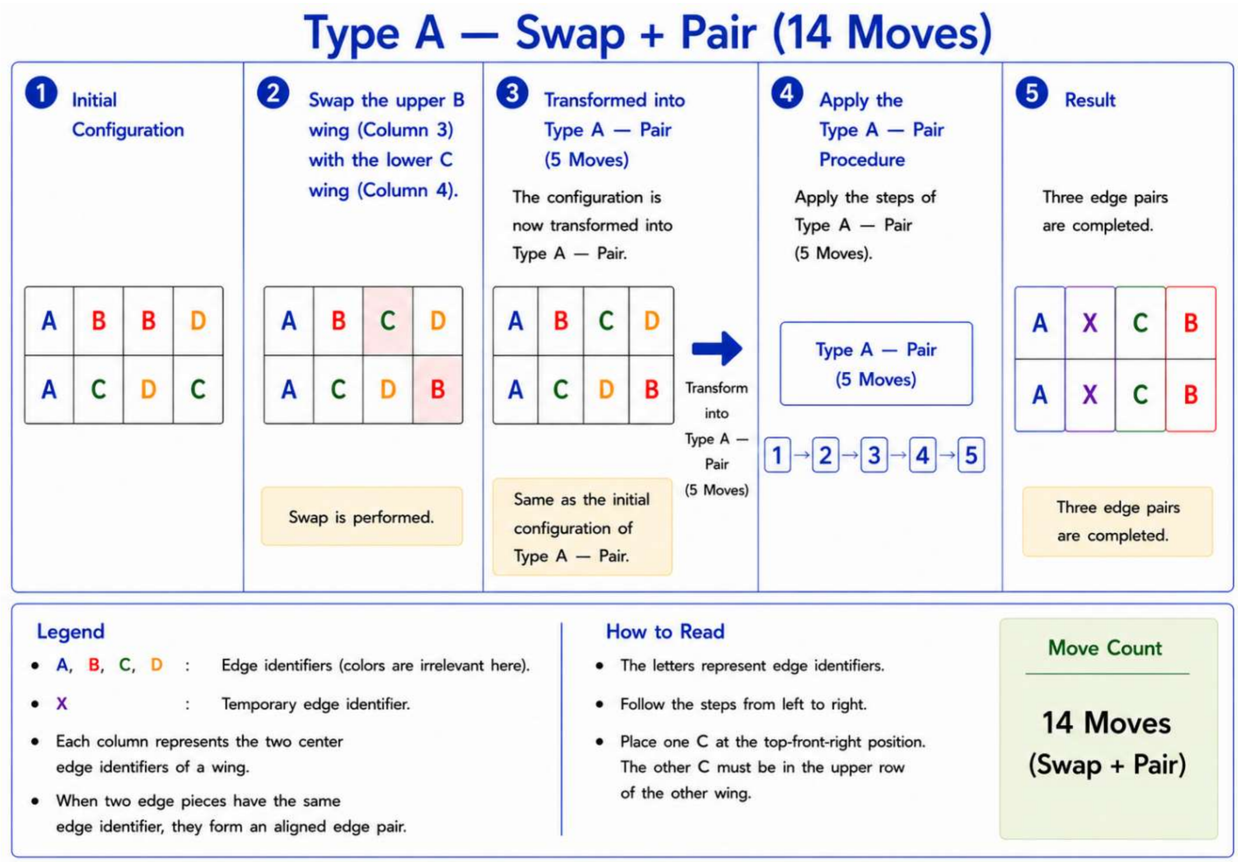

Figure 4.9: Type A — Swap + Pair (14 Moves).

7.2 Type B — Swap + Pair

In Type B, swapping the indicated edge pieces transforms the configuration into **Type B — Pair**. The corresponding five-move pairing procedure then completes all three edge pairs.

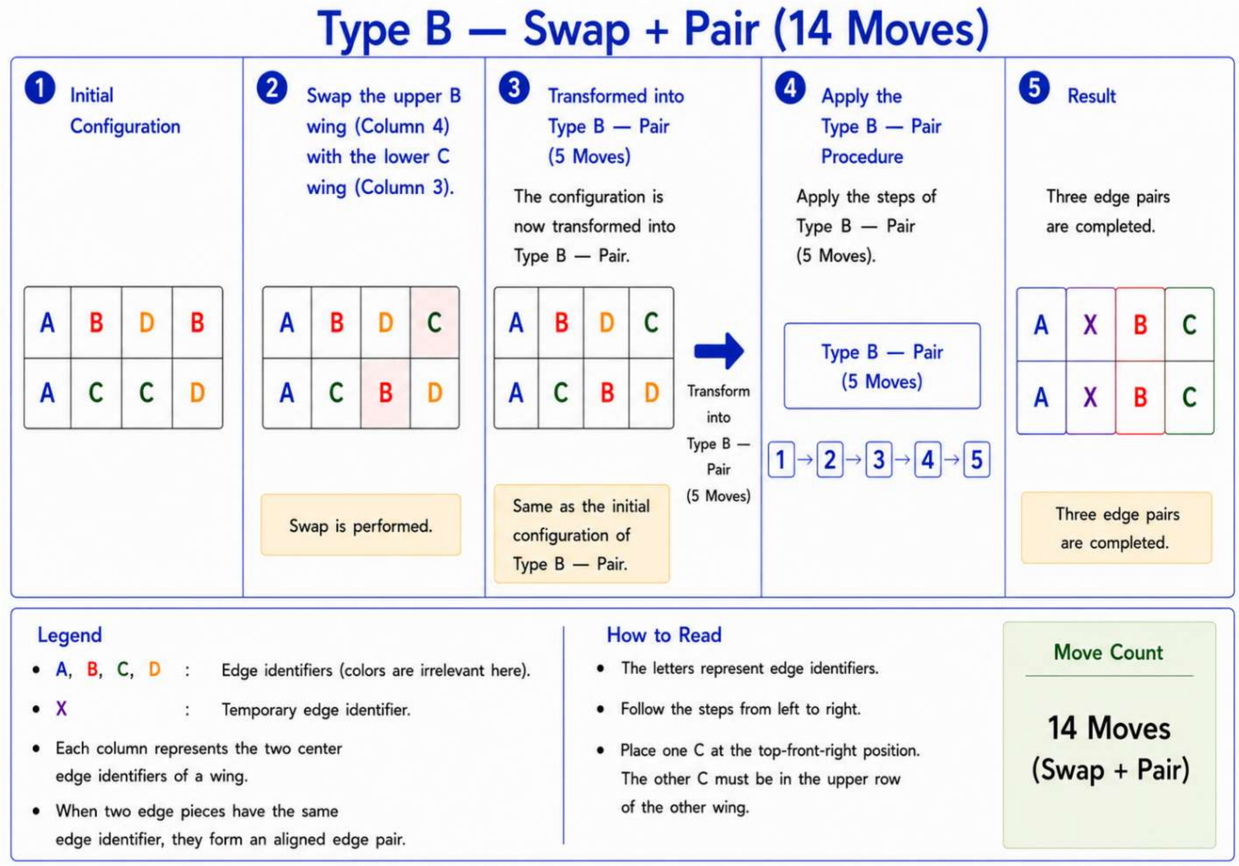

Figure 4.10: Type B — Swap + Pair (14 Moves).

---

8. Summary

The complete pairing strategy can be summarized as follows.

- Perform **Look Ahead**.
- Try **Search Path 1**.
- If both matching pairs are found, apply **Type A** or **Type B**.
- If unsuccessful, try the complementary **Search Path 2**.
- If both matching pairs are found, apply **Type A** or **Type B**.
- Otherwise, perform a **Flip** or **Swap**, then apply **Type A** or **Type B**.
- Repeat the process until all three remaining edge pairs are completed.

By looking ahead before every pairing, this method preserves the **L3E** structure and avoids unnecessarily inducing an **L2E** configuration.

Only **three move counts** need to be remembered throughout Wang's Simple L3E Pairing Method:

- **5 moves** for direct pairing (**Type A** or **Type B**),
- **12 moves** for **Flip + Pair**, and
- **14 moves** for **Swap + Pair**.

Unlike the traditional Yau L3E pairing approach, this method does **not** require memorizing numerous algorithms or exceptional cases. Instead, it combines two complementary search paths with four simple procedures—**Type A**, **Type B**, **Flip**, and **Swap**—to provide a simple, systematic, and easy-to-learn solution while preserving the principal advantage of the Yau edge pairing method: the solved white cross is maintained, allowing the solver to proceed directly to the 3×3 **CFOP** stage without rebuilding the white cross.

Acknowledgments

The author gratefully acknowledges the Rubik's Cube community for generously sharing educational videos, tutorials, and solving techniques online. Watching many of these videos provided valuable opportunities to observe different solving styles and inspired further exploration of simpler and more systematic approaches to the L3E pairing problem.

Wang's Simple L3E Pairing Method presented in this notebook was independently developed through the author's own analysis, classification, and optimization of L3E configurations.

References

[1] Feliks Zemdegs, *Yau Method for Solving the 4x4 Cube*, CubeSkills, 2020.

[2] J Perm, *Learn How to Solve a 4x4 in 10 Minutes (Full Yau Method Tutorial)*, YouTube, January 29, 2018.

[3] J Perm, *4x4 Yau Method: 3-2-3 Edge Pairing (+ Special Cases)*, YouTube, 2017.

[4] Noah Richardson, *4×4 Last Edges Pairing (Beginner Friendly) | Final Edge Fix Explained*, YouTube, January 2026.

The Wang Method Series

Notebook #5 (Version 1.0)

Structural Classification and State Transition of the 24 Top-Layer Four-Edge-Pair Patterns

A Structural Framework for Understanding Top-Layer Four-Edge-Pair Patterns

Laung-Terng (LT) Wang

August 1, 2026

---

1. Introduction

The four top-layer edge pairs can be arranged in 24 (= 4!) distinct clockwise permutations. Although these permutations appear quite different, they exhibit a surprisingly simple structural organization.

In this notebook, the solved configuration **R B O G** is defined as the **Golden Pattern**.

Rather than studying each permutation individually, we first classify all 24 top-layer four-edge-pair configurations according to the number and relative positions of their aligned edge pairs. The resulting classification consists of six structural groups. We then show that every illegal state can always be transformed into a legal state through one of two deterministic transition paths.

This notebook complements the practical solution methods presented in the previous notebooks by providing a unified structural perspective.

---

2. The 24 Top-Layer Four-Edge-Pair Patterns

The 24 possible clockwise permutations are listed in Table 5.1.

Table 5.1. The 24 Top-Layer Four-Edge-Pair Patterns.

Case	Pattern (Clockwise)
1	R B O G
2	R B G O
3	R O B G
4	R O G B
5	R G B O
6	R G O B
7	B R O G
8	B R G O
9	B O R G
10	B O G R
11	B G R O
12	B G O R
13	O R B G
14	O R G B
15	O B R G
16	O B G R
17	O G R B
18	O G B R
19	G R B O
20	G R O B
21	G B R O
22	G B O R
23	G O R B
24	G O B R

Although these 24 patterns appear unrelated, they can be completely classified into only six structural groups.

---

3. Structural Classification of All 24 Patterns

Figure 5.1 illustrates the structural classification of all 24 patterns.

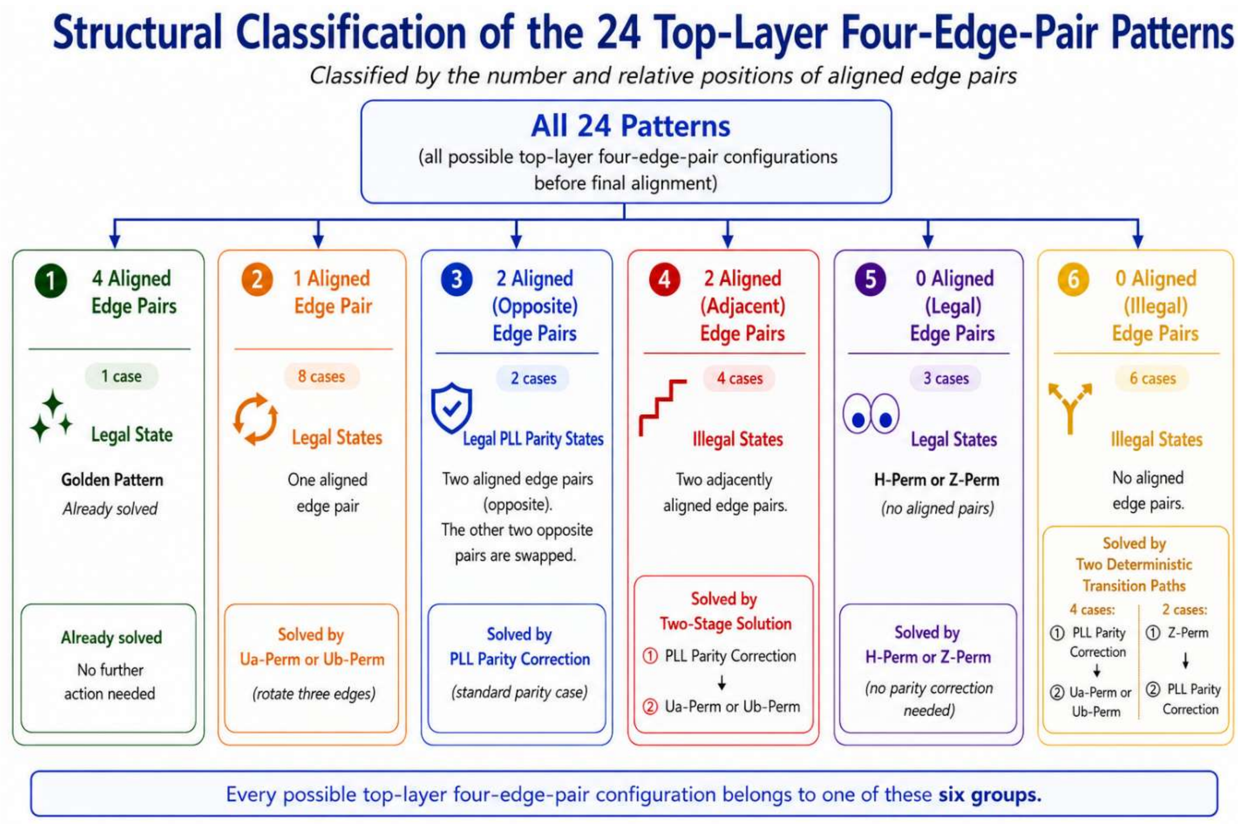

Figure 5.1. Structural classification of the 24 top-layer four-edge-pair patterns before final alignment.

The complete classification and corresponding solution methods are summarized in Table 5.2.

Table 5.2. Classification and Solution of All 24 Patterns.

No. of Aligned Edge Pairs	State Type	Case	Pattern (Clockwise)	Solution
4	Golden (Solved)	1	R B O G	Golden Pattern
1	Legal	4	R O G B	*Ua-Perm*
1	Legal	9	B O R G	*Ua-Perm*
1	Legal	12	B G O R	*Ua-Perm*
1	Legal	16	O B G R	*Ua-Perm*
1	Legal	20	G R O B	*Ua-Perm*
1	Legal	21	G B R O	*Ua-Perm*
1	Legal	5	R G B O	*Ub-Perm*
1	Legal	13	O R B G	*Ub-Perm*
2	Legal (Parity)	6	R G O B	PLL Parity Correction (BG)
2	Legal (Parity)	15	O B R G	PLL Parity Correction (OR)
2	Illegal	2	R B G O	Parity (BO) → R O G B → *Ua-Perm*
2	Illegal	3	R O B G	Parity (OG) → R G B O → *Ua-Perm*
2	Illegal	7	B R O G	Parity (RG) → B G O R → *Ua-Perm*
2	Illegal	22	G B O R	Parity (BR) → G R O B → *Ua-Perm*
0	Legal	17	O G R B	*H-Perm* (OR, GB)
0	Legal	8	B R G O	*Z-Perm* (BR, GO)
0	Legal	24	G O B R	*Z-Perm* (GR, OB)
0	Illegal	11	B G R O	Parity (GO) → B O R G → *Ua-Perm*
0	Illegal	14	O R G B	Parity (RB) → O B G R → *Ua-Perm*
0	Illegal	18	O G B R	Parity (OB) → B G O R → *Ua-Perm*
0	Illegal	23	G O R B	Parity (GR) → R O G B → *Ua-Perm*
0	Illegal	10	B O G R	*Z-Perm* (BR, OG) → R G O B → Parity (GB)
0	Illegal	19	G R B O	*Z-Perm* (GR, BO) → R G O B → Parity (GB)

The 24 patterns are classified into the following six groups.

Group 1 — Golden Pattern

Only one pattern has four aligned edge pairs. This is the solved state and is defined as the Golden Pattern.

Group 2 — One Aligned Edge Pair

Eight patterns have one aligned edge pair. These are all legal states and can be solved directly by either a *Ua-Perm* or a *Ub-Perm*.

Group 3 — Two Opposite Edge Pairs Aligned

Two patterns belong to this group. These are the standard legal PLL parity states and can be solved directly by a standard PLL parity correction.

Group 4 — Two Adjacent Edge Pairs Aligned

Four patterns have two adjacent edge pairs aligned. These are illegal states and require a two-stage solution consisting of a PLL parity correction followed by either a *Ua-Perm* or a *Ub-Perm*.

Group 5 — No Aligned Edge Pairs (Legal States)

Three patterns have no aligned edge pairs but are nevertheless legal states. One pattern is solved by an *H-*Perm, and the other two patterns are solved by a *Z-Perm*.

Group 6 — No Aligned Edge Pairs (Illegal States)

Six patterns have no aligned edge pairs and are illegal states.

Four patterns are transformed into legal *Ua* or *Ub* states through a PLL parity correction, whereas the remaining two patterns first require a *Z-Perm* before reaching legal PLL parity states.

3.1 Key Observations

Several important observations can now be made.

First, the number of aligned edge pairs alone does not determine whether a state is legal.

For example,

- two opposite edge pairs aligned always form legal PLL parity states;
- two adjacent edge pairs aligned always form illegal states;
- no aligned edge pairs may represent either legal or illegal states.

Therefore, both the number and the relative positions of aligned edge pairs are essential for determining legality.

Furthermore, a PLL parity correction may produce either a legal *Ua* state or a legal *Ub* state, depending on which opposite edge pair is selected during the parity correction. These two transition paths are completely symmetric.

Together, these observations partition all 24 top-layer four-edge-pair configurations into six structural groups and provide a unified framework for understanding their legality and solution methods.

---

4. Deterministic State Transition of the 10 Illegal States

Among the 24 patterns, 10 are illegal states.

Fortunately, every illegal state can always be transformed into a legal state through one of two deterministic transition paths. Figure 5.2 summarizes these two transition paths.

Eight illegal states are transformed into legal states through a PLL parity correction. Depending on the selected opposite edge pair during the parity correction, the resulting legal state becomes either a *Ua* state or a *Ub* state. The corresponding *Ua-Perm* or *Ub-Perm* then returns the cube to the Golden Pattern.

The remaining two illegal states first require a *Z-Perm*, which converts them into standard legal PLL parity states. A standard PLL parity correction then completes the solution.

Therefore, every illegal state follows one of only two deterministic transition paths.

Most importantly, every illegal state is transformed into a legal state in exactly two stages.

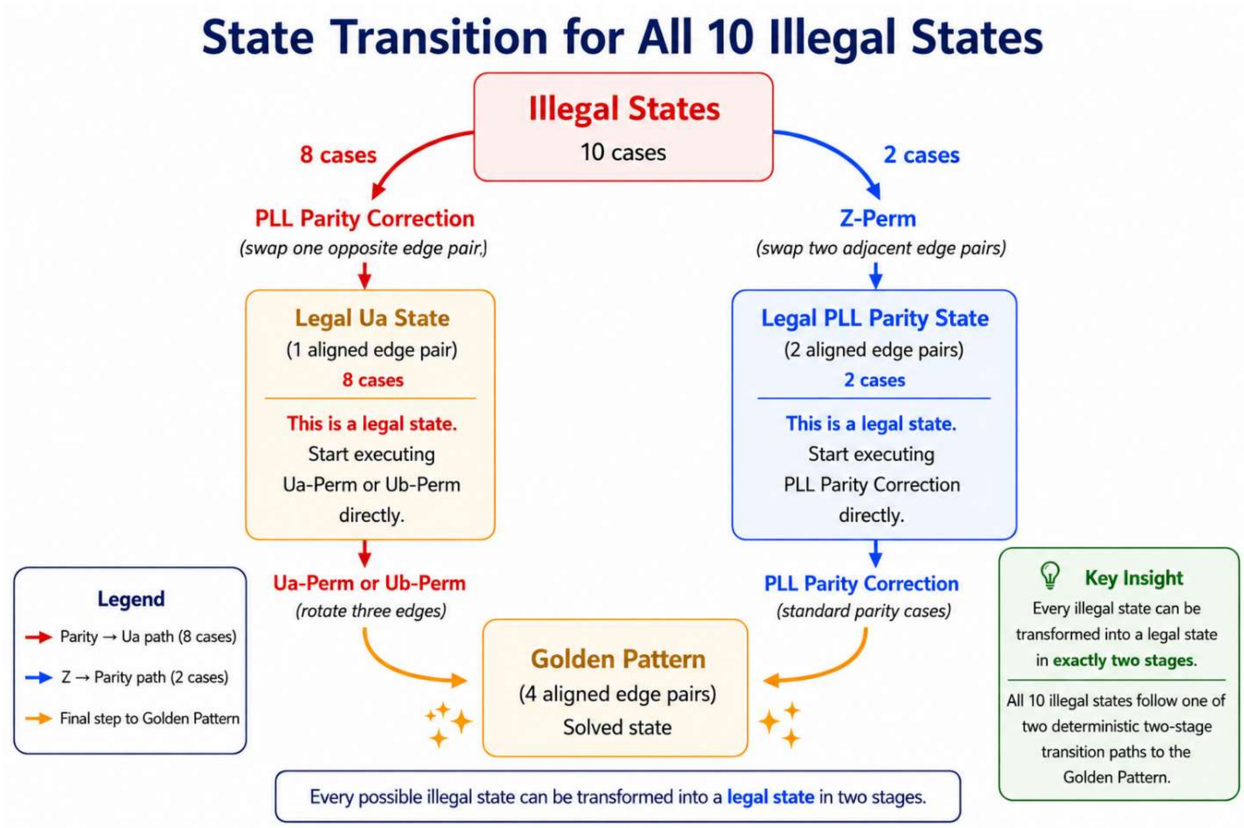

Figure 5.2. State transition of all 10 illegal states

---

5. Conclusions

This notebook presents a complete structural classification of all 24 top-layer four-edge-pair patterns.

The major findings are summarized below.

1. The 24 patterns are completely classified into six structural groups.
2. The legality of a pattern depends on both the number and the relative positions of its aligned edge pairs.
3. Two opposite edge pairs aligned always form legal states, whereas two adjacent edge pairs aligned always form illegal states. States with no aligned edge pairs may be either legal or illegal.
4. Every illegal state can always be transformed into a legal state in exactly two deterministic stages.
5. A PLL parity correction may produce either a legal *Ua* state or a legal *Ub* state, depending on the selected opposite edge pair. The two transition paths are completely symmetric.
6. Together, the structural classification and the state-transition framework provide a complete theoretical understanding of the 24 top-layer four-edge-pair patterns and complement the practical solution methods presented in the previous notebooks.

Although different intermediate legal states (*Ua* or *Ub*) may be reached, every illegal state ultimately follows one of two deterministic transition paths back to the Golden Pattern.